Assignment 2

Part I

Q1) The query returns 0 because NULL represents an unknown values and comparing it with = always results in unknown, not TRUE. Since WHERE only keeps rows where the condition is true, rows with phone_number = NULL never get counted, even if phone_number actually is NULL. To check for missing values SQL uses the IS NULL operator

Corrected query: SELECT COUNT(*) FROM Student WHERE phone_number IS NULL;

Aggregate functions usually ignore NULL values because they represent missing data, and comparision operators cannot evaluate unknown values

Q2) Yes, the student is correct. GROUP BY department already produces one row per department. Since each department appears only once after grouping, DISTINCT has no effect of removing duplicate rows and is unnecessary in this query

Q3) If the LEFT JOIN returns more rows than the INNER JOIN, it means some customers do not have matching orders. INNER JOIN returns only customers with at least one order, whereas LEFT JOIN returns all customers, including those who have never placed an order. The extra rows in the LEFT JOIN contain NULL values for the order-related columns, indicating customers without any matching orders

Say the Customer table has 100 customers, but 10 of them signed up and never placed an order. INNER JOIN would only return rows for the 90 customers who actually have orders (matching each customer to each of their orders). LEFT JOIN would return those same 90 customers' order rows, plus one extra row for each of the 10 customers with no orders (with order_id, restaurant_id, etc. shown as NULL). So LEFT JOIN ends up with at least 10 more rows than INNER JOIN.

Q4) The query is invalid because, GROUP BY department collapses all employees in the same department into a single row. But employee_name is neither part of the GROUP BY nor wrapped in an aggregate function, so SQL has no rule for picking which employee's name to show when a department has multiple employees

Correct query:

SELECT department, AVG(salary)

FROM Employee

GROUP BY department;

If allowed: SQL would have to arbitrarily choose one name out of possibly several, with no logical basis for the choice that's not a meaningful result, just a guess, which is why SQL blocks it

Q5) **The error:** The student used WHERE AVG(salary) > 80000, but WHERE cannot filter on aggregate functions. SQL processes clauses in a specific order — FROM → WHERE → GROUP BY → HAVING → SELECT. WHERE runs *before* grouping happens, so at that point individual rows exist, not grouped/aggregated values AVG(salary) doesn't exist yet for WHERE to check. Aggregates are only computed during/after GROUP BY, so filtering on them must happen in HAVING, which runs after grouping

Correct Query:

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department
HAVING AVG(salary) > 80000;
```

Q6)

(a) Products priced above the overall average product price -- Non-correlated subquery.
The average price here is a single fixed value computed once from the whole Product table, and it doesn't depend on each outer row. So the subquery can run independently, just once.
Query:

```
SELECT * FROM Product
WHERE price > (SELECT AVG(price) FROM Product);
```

(b) Employees earning more than their own department's average salary — Correlated subquery.
Here the "average" is different for each department, so the subquery must reference the outer employee's department and be re-evaluated for every row
Query:

```
SELECT * FROM Employee e1
WHERE salary > (
  SELECT AVG(salary) FROM Employee e2
  WHERE e2.department = e1.department
);
```

Key performance implication:
A correlated subquery re-runs once per row of the outer table, so on a large outer table it can become very slow (essentially nested-loop behavior). A non-

correlated subquery runs only once, regardless of outer table size, so it's much cheaper

Part II

a) SELECT c.customer_id, c.name, c.email, c.phone

FROM Customer c

LEFT JOIN Orders o

ON c.customer_id = o.customer_id

WHERE o.order_id IS NULL;

A LEFT JOIN is used to include all customers, even those who have not placed any orders. Customers with no matching order have NULL in the Orders table, so WHERE o.order_id IS NULL identifies them

b) SELECT restaurant_id,

AVG(order_value) AS avg_order_value

FROM (

SELECT o.order_id,

o.restaurant_id,

SUM(m.price * oi.quantity) AS order_value

FROM Orders o

JOIN OrderItem oi

ON o.order_id = oi.order_id

JOIN MenuItem m

ON oi.item_id = m.item_id

GROUP BY o.order_id, o.restaurant_id

) AS OrderValues

GROUP BY restaurant_id

HAVING COUNT(*) > 5;

The inner query calculates the total value of each order using SUM(price × quantity). The outer query computes the average order value for each restaurant and uses HAVING to include only restaurants with more than five orders

c) SELECT restaurant_id, AVG(order_value) AS avg_order_value

FROM (

  SELECT o.order_id,

      o.restaurant_id,

      SUM(m.price * oi.quantity) AS order_value

  FROM Orders o

  JOIN OrderItem oi

   ON o.order_id = oi.order_id

  JOIN MenuItem m

   ON oi.item_id = m.item_id

  GROUP BY o.order_id, o.restaurant_id

) AS OrderValues

GROUP BY restaurant_id

HAVING AVG(order_value) >

(

  SELECT AVG(order_value)

  FROM (

    SELECT o.order_id,

        SUM(m.price * oi.quantity) AS order_value

    FROM Orders o

    JOIN OrderItem oi

     ON o.order_id = oi.order_id

    JOIN MenuItem m

     ON oi.item_id = m.item_id

    GROUP BY o.order_id

) AS PlatformAverage

);

A non-correlated subquery is used because the platform-wide average order value is calculated only once. The HAVING clause compares each restaurant's average order value with this overall average

d) SELECT m.restaurant_id,

```
    oi.item_id,
    m.item_name,
    SUM(oi.quantity) AS total_quantity
FROM OrderItem oi
JOIN MenuItem m
ON oi.item_id = m.item_id
GROUP BY m.restaurant_id, oi.item_id, m.item_name
HAVING SUM(oi.quantity) = (
  SELECT MAX(total_qty)
  FROM (
    SELECT SUM(oi2.quantity) AS total_qty
    FROM OrderItem oi2
    JOIN MenuItem m2
    ON oi2.item_id = m2.item_id
    WHERE m2.restaurant_id = m.restaurant_id
    GROUP BY oi2.item_id
  ) AS T
);
```

The query finds the total quantity ordered for each menu item and returns the item(s) with the highest quantity for each restaurant. A limitation of using GROUP BY and MAX() is that handling ties or retrieving the top N items becomes complex; window functions like ROW_NUMBER() or RANK() are generally better for such cases

Part III

Q1) The functional dependencies are:

student_id → student_name

course_id → course_name, instructor_id

instructor_id → instructor_name, instructor_office

(student_id, course_id) → grade, enrollment_date

Q2) Yes, the table is in 1NF because each attribute stores only a single (atomic) value, and there are no repeating groups or multivalued attributes. Each row represents one student's enrollment in one course

Q3) The partial dependencies are student_id → student_name and course_id → course_name, instructor_id because they depend only on part of the composite key (student_id, course_id). This causes update anomalies; for example, changing a course name requires updating every enrollment record for that course

Q4) The transitive dependency is course_id → instructor_id → instructor_name, instructor_office. This violates 3NF because instructor details depend on another non-key attribute (instructor_id). If an instructor's office changes, it must be updated in multiple rows, leading to inconsistency

Q5) Student (student_id(PK), student_name)
FD: student_id → student_name

Course (course_id(PK), course_name, instructor_id(FK))
FD**:** course_id → course_name, instructor_id

Instructor (instructor_id(PK), instructor_name, instructor_office)
FD**:** instructor_id → instructor_name, instructor_office

Enrollment (student_id(FK,PK)**,** course_id(FK,PK), grade, enrollment_date)
FD**:** (student_id, course_id) → grade, enrollment_date

Part IV

JOIN vs. Subquery Choice: For finding restaurants whose average order value beats the platform average (Part II-c), I could have used a JOIN against a precomputed platform-average value instead of a subquery. But the platform average is a single number, not a row-matched value joining it in would mean attaching the same repeated value to every restaurant row just to compare against it, which adds an unnecessary join for no real benefit. A subquery computes that one value directly and compares it cleanly, so it was the more natural and readable choice here, with no performance cost since it only runs once.

Normalization Trade-offs: 3NF vs. Denormalization: Once the Enrollment table is split into Student, Course, Instructor, and Enrollment, a query like "show student name, course name, and instructor's office" now needs three or four joins instead of a single

SELECT. This adds query complexity and can slow down reads on large datasets if joins aren't well indexed.

Real systems often accept this trade-off deliberately. Reporting and analytics systems commonly use denormalized schemas to avoid expensive joins at query time. Even in regular transactional systems, teams sometimes keep a denormalized read-optimized copy alongside the normalized tables, trading some redundancy for faster, simpler queries.